F24-089-D-VoiceArchi

Project Team

Haseeb Ali	19I-1889
Faiq Ahmed	21I-0759
Junaid Maqbool	20I-0895

Session 2024-2025

Supervised by

Mr. Irfan Ullah

Co-Supervised by

-

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

October, 2024

Contents

List of Figures

List of Tables

Chapter 1

Introduction

VoiceArchi is a web/mobile application that transforms your spoken ideas into detailed architectural drawings. It aims to revolutionize the process of floorplan creation by introducing a voice-controlled interface, making the creation of 2D floorplans accessible and intuitive. By integrating state-of-the-art AI technologies such as speech recognition and natural language processing, VoiceArchi allows users to simply describe their desired floorplan and have the system automatically generate it based on constraints and relationships derived from their input. This innovation opens new doors for people with limited architectural experience, providing them with a tool that simplifies the process of floorplan creation.**?**.

1.1 Problem Statement

Creating a custom floorplan is often a time-consuming and complex process, requiring specialized knowledge and experience with architectural design software. The laymen who lack the knowledge and experience necessary for creating architectural drawings, using the existing softwares can seem daunting and unintuitive, especially when users have a specific vision in mind. Manual processes also leave room for misinterpretation, which can lead to costly errors and delays. Moreover, while many modern tools allow users to select pre-existing templates, they fail to provide true customization based on natural, spoken descriptions. The challenge, therefore, lies in developing an intelligent system that can bridge the gap between a user's spoken intentions and a detailed, accurate floorplan representation.

1.2 Scope

The scope of VoiceArchi encompasses the development of a web-based and mobile application designed to simplify the process of floorplan creation using voice commands. The primary functionality of the system is to allow users to describe their desired floorplans in natural language, which will be converted to text through Whisper's speech recognition capabilities. Once the voice input is transcribed, the system will extract key architectural constraints such as room types, quantities, sizes, and spatial relationships using a fine-tuned GPT model. These extracted constraints will then be processed through a constraint satisfaction algorithm to generate a detailed 2D floorplan, adhering to the user's specifications.

The scope includes the development of a user-friendly interface for both the web and mobile applications, where users can provide voice input and view the generated floorplans. On the backend, the scope covers the implementation of Whisper for accurate speech-to-text conversion, the fine-tuning of GPT for domain-specific constraint extraction, and the development of the constraint satisfaction algorithm to produce accurate floorplans.

Key Features:

- **Voice Command Input**: Allows users to describe floorplans using natural language voice commands.
- **Real-time Text Conversion**: Converts voice input into text using Whisper for accurate transcription.
- **Constraint Extraction**: Extracts architectural constraints and relationships from the transcribed text using a fine-tuned GPT model.
- **2D Floorplan Generation**: Produces detailed 2D floorplans based on the extracted constraints and user specifications.
- **User-friendly Interfaces**: Provides accessible interfaces for both web and mobile platforms, allowing easy interaction and customization.

1.3 Modules

1.3.1 Client Web App

This module represents the user interface for accessing VoiceArchi via a web browser. It allows users to provide voice input, view real-time transcriptions, and visualize the generated 2D floorplans based on their descriptions.

1. Voice Command Interface: Users can input their requirements using voice.
2. Real-time Text Conversion: Displays transcribed text from voice input using Whisper.
3. Generated 2D Floorplan View: Renders the floorplan based on user input.

1.3.2 Client Mobile App

This module focuses on delivering a mobile interface for VoiceArchi by porting the existing web app functionality into a mobile environment using React Native. It maintains consistency with the web app's features without additional mobile-specific optimizations.

1. Voice Command Input: Provides the same functionality as the web app, allowing users to input their requirements using voice commands.
2. Mobile Floorplan Rendering: Displays the generated floorplan in a view adapted for mobile screens, ensuring usability on various mobile devices.

1.3.3 Backend Processing

This module handles the core processing tasks, including voice-to-text conversion, constraint extraction, and floorplan generation.

1. Whisper-based Speech Recognition: Converts voice input into text with high accuracy.
2. Constraint Extraction with Fine-tuned GPT: Extracts architectural constraints and relationships (e.g., number of rooms, adjacency) from transcribed text. Using node.js to integrate python with our web/mobile application.
3. Constraint Satisfaction Algorithm: Generates a 2D floorplan based on the extracted constraints and user preferences.

1.4 User Classes and Characteristics

Identify the various user classes that you anticipate will use this product, and describe their pertinent characteristics.

User class	Description
Architects	Professional architects who can use the platform to quickly generate initial floorplan sketches from verbal descriptions. These users are experienced with architectural design but may want a faster and more efficient way to create draft floorplans without manually drawing them.
Interior Designers	Interior designers can use VoiceArchi to understand the spatial arrangement and visualize room layouts based on verbal client requirements. The tool allows them to create mockups or design plans quickly.
Homeowners and Clients	Individuals who are looking to design or modify their homes can use VoiceArchi to communicate their ideas more effectively. These users are typically non-technical and may not be familiar with formal design tools, so ease of use and intuitive voice interactions are crucial for this class.

Example: User Classes and Characteristics

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

2.1.1 Create Floorplan using Voice Commands

Actors: End User

Preconditions:

- The user is logged into the system.
- The application is connected to the voice recognition system.

Trigger: The user initiates the voice command mode.

Basic Flow:

- The user activates the voice command input.
- The system prompts the user to describe the floorplan.
- The user provides voice commands detailing rooms and constraints.
- Whisper converts voice to text.
- The system processes the text and extracts constraints.
- A constraint satisfaction algorithm generates the floorplan.
- The system displays the 2D floorplan for review.

- The user accepts or requests adjustments.

Postconditions:

- A floorplan is generated and displayed.
- The system saves the design for further modifications.

Exceptions:

- If voice recognition fails, the user is prompted to retry or use text input.

2.1.2 View Your Designs

Actors: End User

Preconditions:

- The user has logged into the system.

Trigger: The user selects the option to view saved designs.

Basic Flow:

- The system displays a list of previously saved designs.
- The user selects a design to view.
- The system shows the 2D layout of the selected design.

Postconditions:

- The selected design is displayed to the user.

2.1.3 Save Design

Actors: End User

Preconditions:

- A floorplan has been created or modified.

Trigger: The user selects the option to save the design.

Basic Flow:

- The user clicks the "Save" button.
- The system saves the current design under a specific name or timestamp.

Postconditions:

- The design is saved and stored for future use.

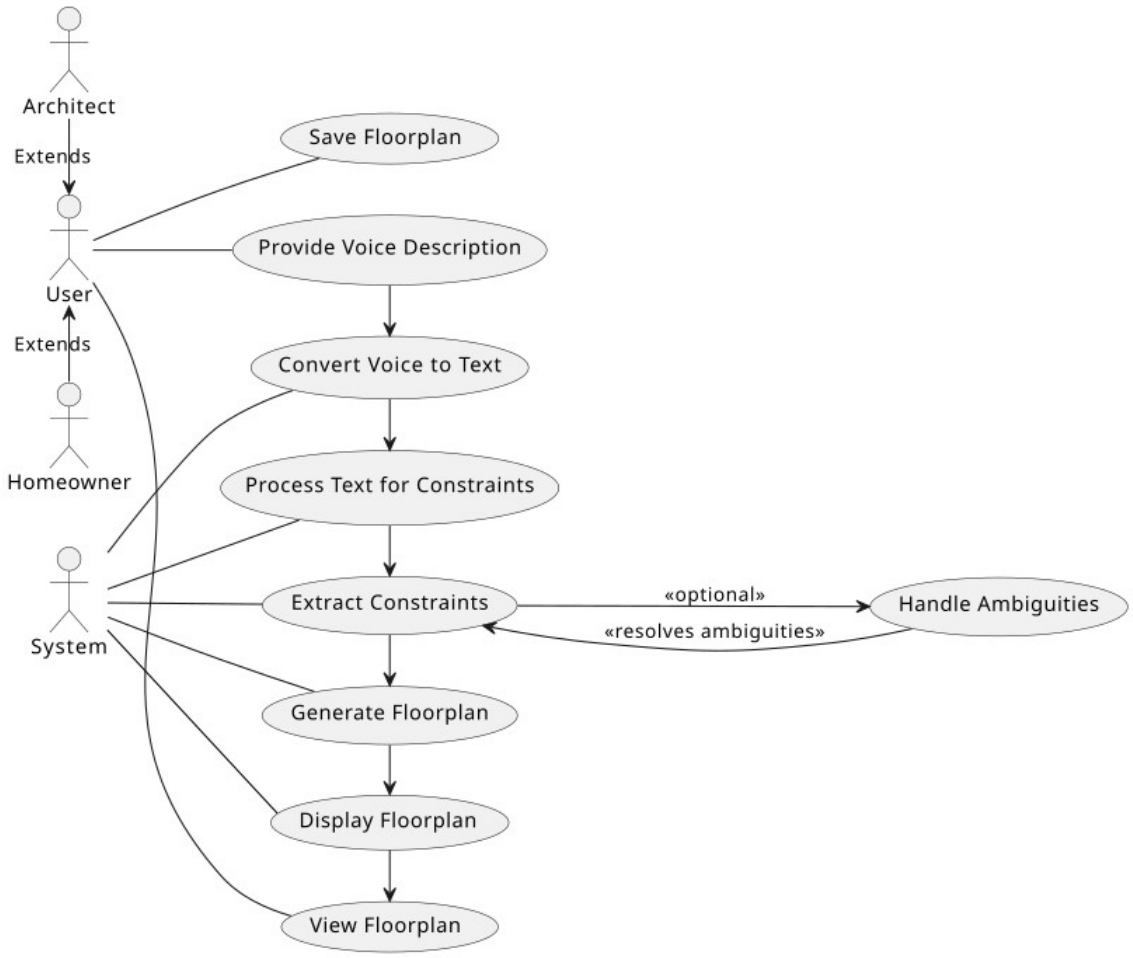

Figure 2.1: Use Case Diagram

2.2 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. The requirements are organized by feature within each module. These features describe the expected behavior of the system for each module.

2.3 Modules

2.3.1 Client Web App

Following are the functional requirements for the web interface that allows users to interact with VoiceArchi:

1. **FR1: Voice Command Input**
   The system must allow the user to activate and provide voice input through the browser interface.
2. **FR2: Real-time Text Conversion**
   The system must display the real-time transcriptions of the user's voice commands using Whisper.
3. **FR3: Generated 2D Floorplan View**
   The system must render and display the generated 2D floorplan based on the extracted constraints from user input.

2.3.2 Client Mobile App

Following are the functional requirements for the mobile interface, which ports the web app functionality:

1. **FR1: Voice Command Input**
   The system must allow the user to provide voice input via the mobile interface, maintaining functionality consistent with the web app.
2. **FR2: Mobile Floorplan Rendering**
   The system must adapt and display the generated 2D floorplan on mobile devices, ensuring usability and responsiveness on different screen sizes.

2.3.3 Backend Processing

The backend processing handles core functionalities such as voice-to-text conversion, constraint extraction, and floorplan generation. Following are the functional requirements for this module:

1. **FR1: Whisper-based Speech Recognition**
   The system must convert voice input into text using the Whisper model with high accuracy.

2. **FR2: Constraint Extraction with Fine-tuned GPT**
   The system must extract architectural constraints from the transcribed text, including room types, sizes, and adjacency requirements.

3. **FR3: Constraint Satisfaction Algorithm**
   The system must generate a 2D floorplan based on the extracted constraints, ensuring that all user-specified conditions are satisfied.

2.4 Non-Functional Requirements

This section specifies the non-functional requirements of the system. These quality requirements are specific, quantitative, and verifiable, covering various aspects such as reliability, usability, performance, and security.

2.4.1 Reliability

The reliability of the system is crucial to ensure it functions smoothly without frequent failures.

- **REL-1: MTBF (Mean Time Between Failures)**
  The system shall have an MTBF of at least 1000 hours of continuous operation, ensuring long periods of uninterrupted service.

- **REL-2: Failure Response**
  In case of a failure, the system must automatically attempt recovery without data loss. Users should be notified of critical failures and given an option to retry or abort operations.

- **REL-3: Error Detection and Correction**
  The system shall implement error detection mechanisms to identify issues in real-time and attempt self-correction. Logs of all failures shall be stored for future diagnosis.

2.4.2 Usability

The usability of the system should ensure that users, regardless of their technical proficiency, can easily interact with VoiceArchi. This includes ease of learning and the efficiency of interactions.

- **USE-1: Intuitive Interface**
  The system interface must be simple and intuitive, allowing users to create, view, and manage floorplans with minimal effort.
- **USE-2: Voice Command Recognition Accuracy**
  The system shall have a minimum 90% accuracy in recognizing voice commands, ensuring seamless interactions.
- **USE-3: Accessibility**
  The system must comply with accessibility standards, including support for screen readers, adjustable text size, and contrast options for users with disabilities.

2.4.3 Performance

The performance of the system defines how quickly and efficiently the system processes requests and delivers results to the user.

- **PER-1: Response Time for Voice Command Processing**
  95% of voice commands shall be processed and transcribed within 3 seconds.
- **PER-2: Floorplan Generation Time**
  The system shall generate a 2D floorplan within 5 seconds after the user provides all required input constraints.
- **PER-3: Mobile Application Responsiveness**
  The mobile app should respond to user input with less than 200 milliseconds delay to maintain smooth navigation.

2.4.4 Security

The security of the system is paramount to protect user data and ensure safe interactions.

- **SEC-1: Data Encryption**
  All user data, including saved floorplans and personal information, shall be encrypted using AES-256 during both storage and transmission.
- **SEC-2: Authentication**
  The system must support multi-factor authentication (MFA) to ensure secure user login and prevent unauthorized access.
- **SEC-3: System Penetration Resistance**
  The system should be resistant to penetration attempts, requiring at least 48 hours of sustained effort from a highly skilled attacker to breach.

2.5 Domain Model

Create a representation of the domain model for your project.

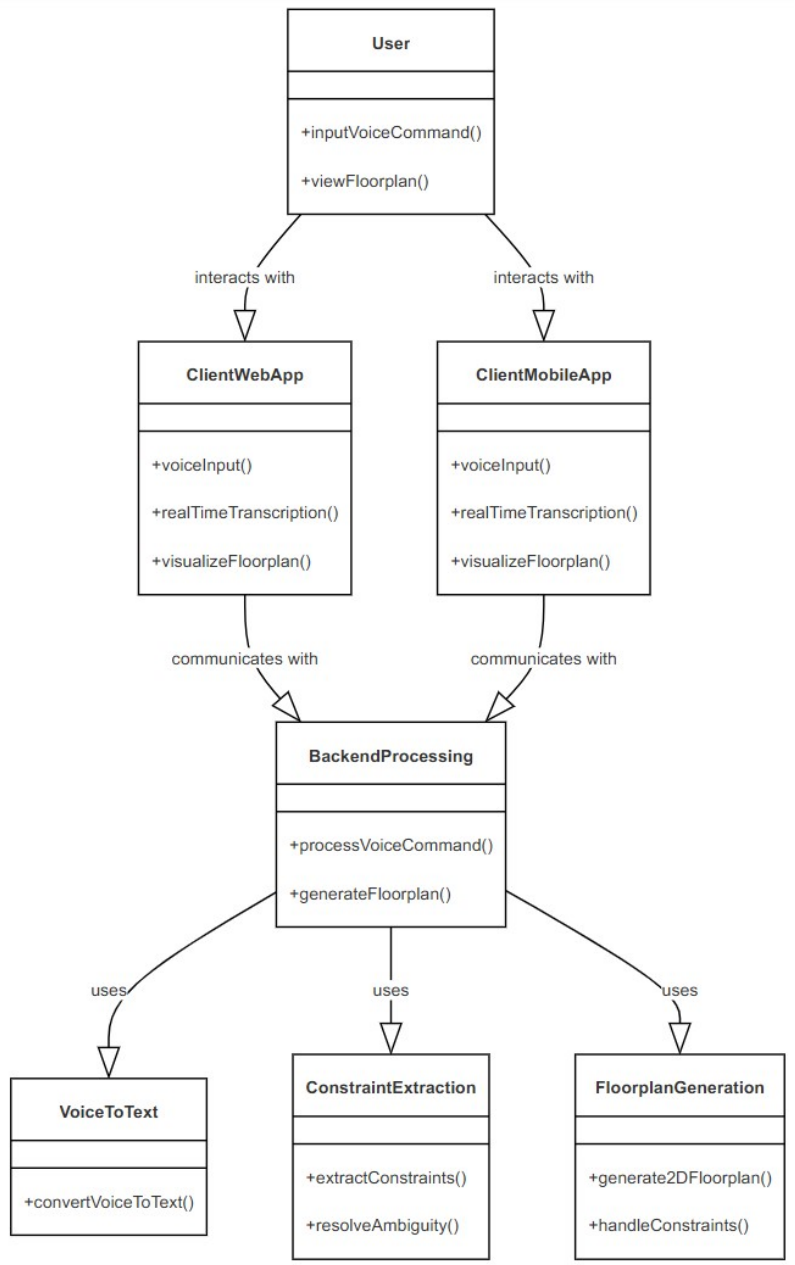

Figure 2.2: Domain Model for VoiceArchi

Chapter 3

System Overview

VoiceArchi is a web application that transforms spoken ideas into detailed 2D floorplans using AI technologies like Whisper for voice-to-text conversion and GPT for constraint extraction. It simplifies the floorplan creation process, making it accessible to users without technical expertise.

Functionality

The core functionality allows users to:

- Create a new floorplan by providing voice input that is transcribed and processed to extract spatial constraints.
- View previously created designs stored in the system.
- Resolve ambiguities through confirmations during the design process.

A constraint satisfaction algorithm generates a 2D floorplan once all constraints are confirmed.

Context

VoiceArchi serves both novice users and professionals, offering an intuitive alternative to traditional architectural tools. It enables rapid prototyping through natural voice input, making floorplan design more accessible.

Design

VoiceArchi is composed of:

- **Client Web App**: A user-friendly interface for voice input, real-time transcription, and 2D floorplan visualization.
- **Backend Processing**: Handles voice-to-text conversion (Whisper), constraint extraction (GPT), and floorplan generation using a constraint satisfaction algorithm.

This modular design ensures a seamless user experience while maintaining scalability.

3.1 Architectural Design

The VoiceArchi system is designed to achieve a modular, maintainable structure where each module handles a specific responsibility, collaborating through defined interfaces. The system primarily follows a client-server architecture with a multi-tiered structure, ensuring separation of concerns and scalability.

The major subsystems and their relationships are described below:

1. **Voice Command Input:** This subsystem captures voice commands from the user. It serves as the entry point for user interaction, providing the necessary inputs for floorplan generation.
2. **Real-time Text Conversion (Whisper):** This module handles the voice-to-text conversion. It takes the voice input and converts it into text in real-time using the Whisper model. The output text is passed to the next stage for processing.
3. **Constraint Extraction (Fine-tuned GPT):** The transcribed text is processed to extract architectural constraints using a fine-tuned GPT model. This includes identifying rooms, dimensions, and relationships between spaces. The constraints are then fed into the floorplan generation module.
4. **2D Floorplan Generation:** Based on the constraints provided, this module generates a 2D floorplan using a constraint satisfaction algorithm. It ensures that the user's specifications, such as room size and adjacency, are adhered to.
5. **User Interfaces (Web & Mobile):** The system supports both web and mobile interfaces, where users can interact with the generated floorplans. Users can also input commands, review designs, and make adjustments.
6. **Data Storage:** This module stores user designs and project data securely. It interacts with the user interfaces to allow saving and loading of designs.
7. **API for Fine-tuned GPT Interaction:** An API facilitates communication with the GPT model for constraint extraction, ensuring modularity and flexibility in updating the model if needed.

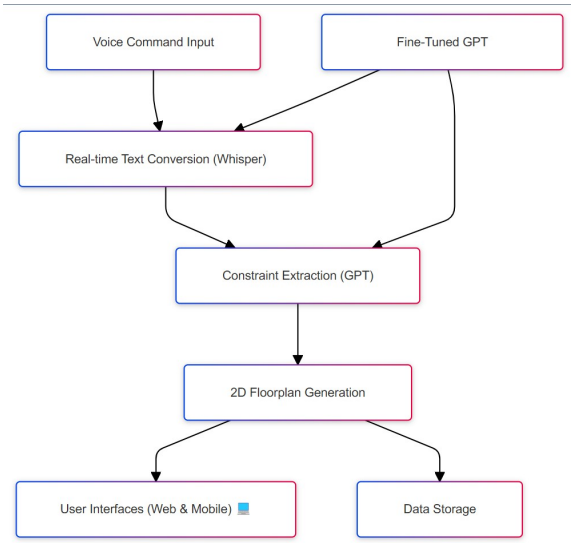

Figure 3.1: Box and Line Diagram

Final Architecture

- **Client-Side (Presentation Layer):** The web and mobile UIs are responsible for providing interaction points for users to give voice commands and visualize the generated floorplans. The voice input module captures the user's commands and sends them to the backend for processing.
- **Backend (Application Layer):** This layer handles the core business logic, including the real-time voice-to-text conversion using Whisper, constraint extraction using GPT, and the final step of floorplan generation using constraint satisfaction algorithms. The backend interacts with the client to send back results (generated floorplans) for display.
- **Data Layer:** This layer stores user-created designs, ensuring persistence across sessions. It also handles the communication between the backend and the fine-tuned GPT model via the API, ensuring modularity and future extensibility.

3.2 Design Models

Create design models as are applicable to your system. Provide detailed descriptions with each of the models that you add. Also ensure visibility of all diagrams.

Design Models for Object Oriented Development Approach

The applicable models for the project using object oriented development approach may include:

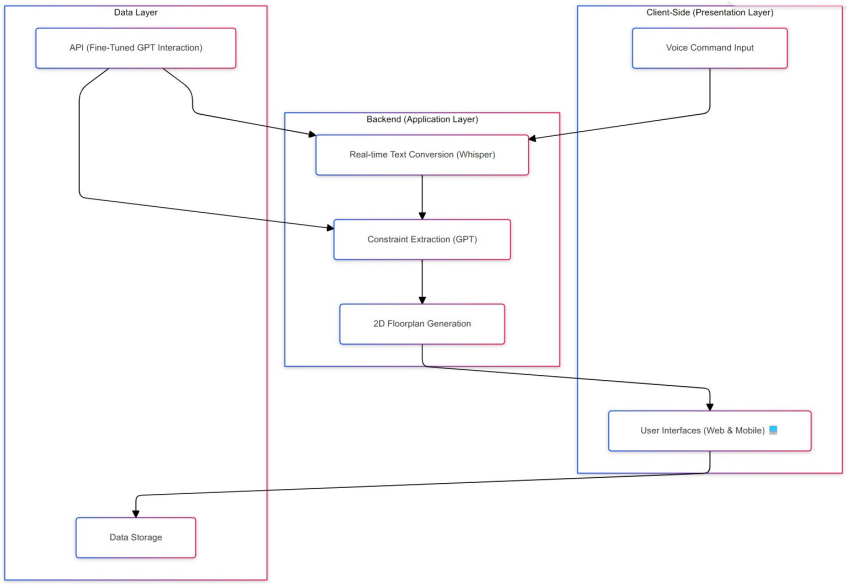

Figure 3.2: Architecture Diagram

• Activity Diagram
• Class Diagram
• Class-level Sequence Diagram
• State Transition Diagram (for the projects which include event handling and backend processes)

Design Models for Procedural Approach

The applicable models for the project using procedural approach may include:

• Activity Diagram
• Data Flow Diagram (data flow diagram should be extended to 2-3 levels. It should clearly list all processes, their sources/sinks and data stores.)
• System-level Sequence Diagram
• State Transition Diagram (for the projects which include event handling and backend processes)

3.3 Data Design

Explain how the information domain of your system is transformed into data structures. Describe how the major data or system entities are stored, processed, and organized.

List any databases or data storage items.

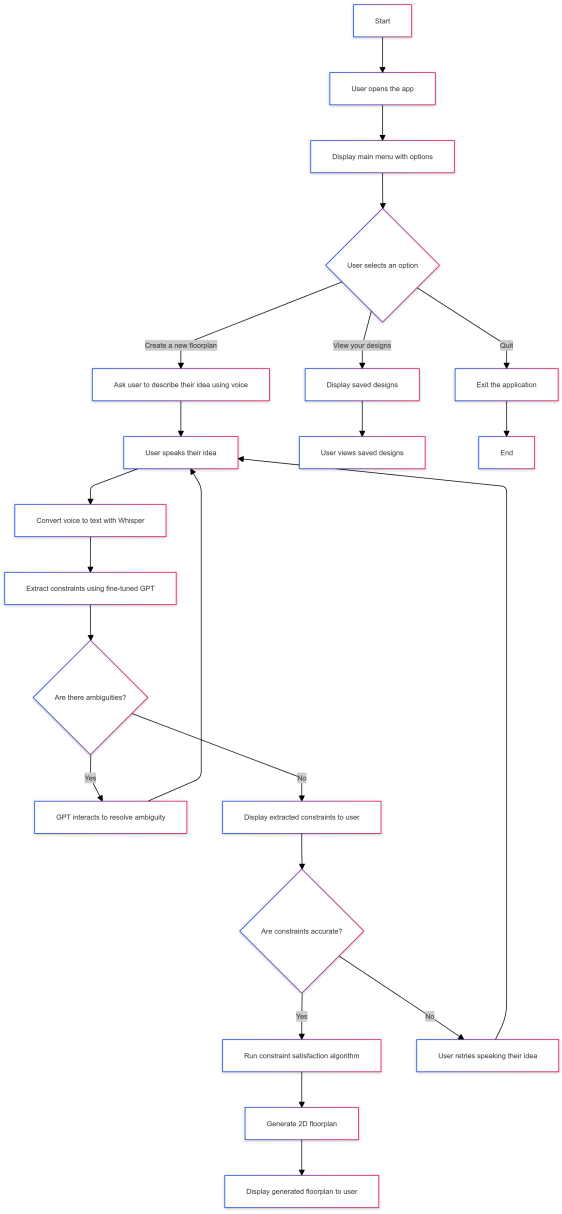

Figure 3.3: Activity Diagram

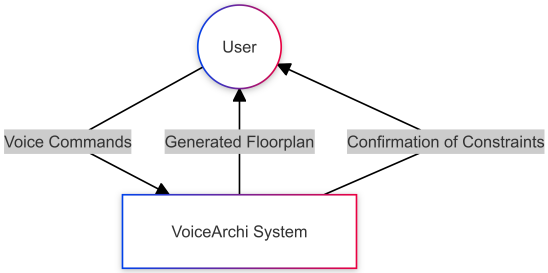

Figure 3.4: Level 0

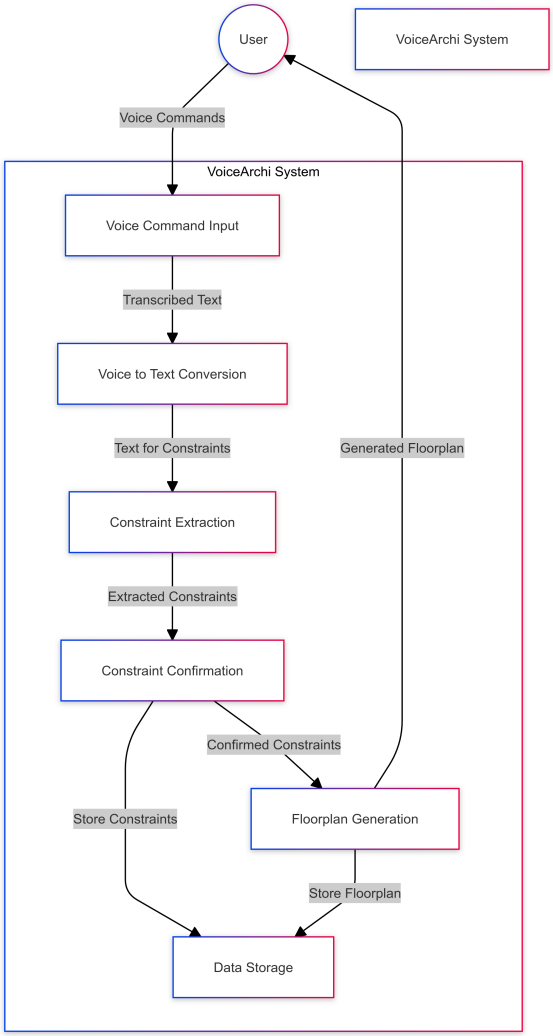

Figure 3.5: Level 1

Entities and Attributes

User

Represents users of the application.

- **Attributes**:
  - `userID`: Unique identifier for each user (Primary Key).
  - `username`: Name of the user.
  - `email`: Email address of the user.

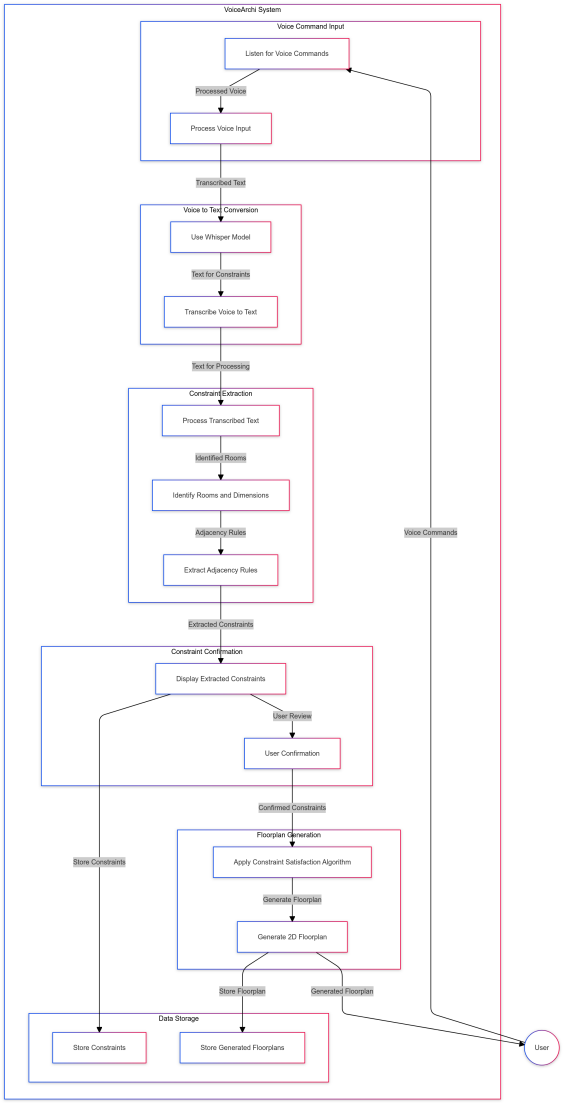

Figure 3.6: Level 2

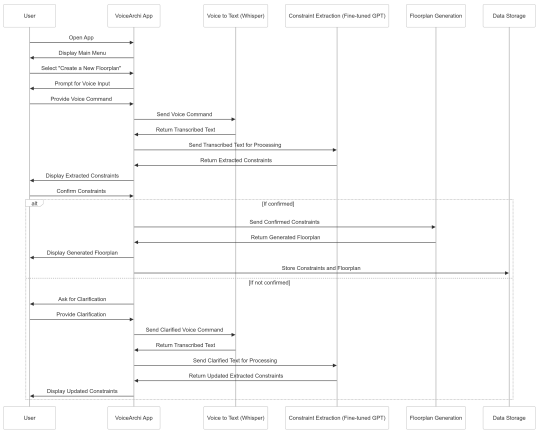

Figure 3.7: Sequence Diagram

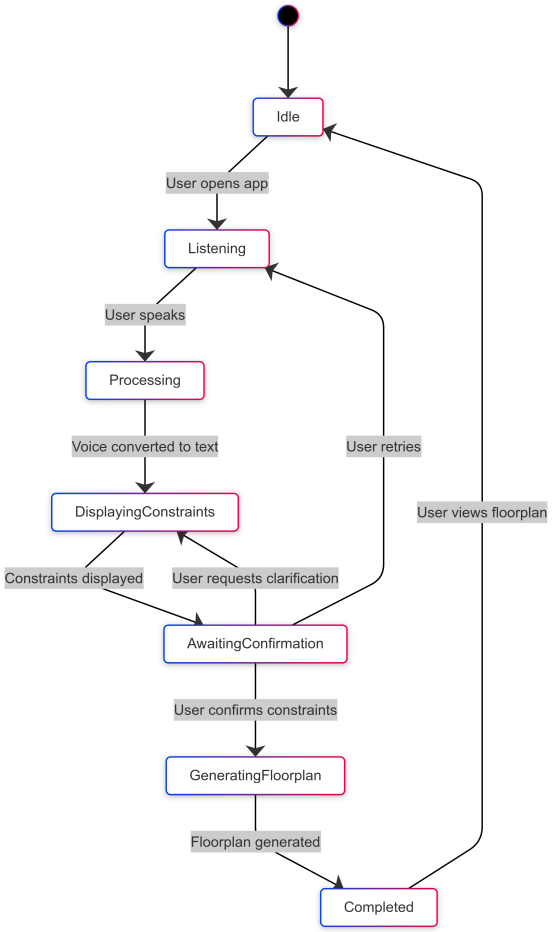

Figure 3.8: State Transition Diagram

Floorplan

Represents the generated floorplans based on user commands.

- **Attributes**:
  - `floorplanID`: Unique identifier for each floorplan (Primary Key).
  - `userID`: Foreign Key linking to the User who created the floorplan.
  - `constraints`: The constraints extracted from the user's input.
  - `designData`: The actual data or representation of the 2D floorplan.

VoiceCommand

Represents the voice commands provided by the users.

- **Attributes**:

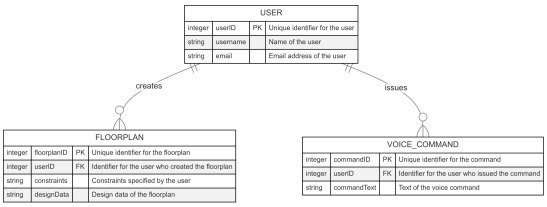

Figure 3.9: ERD Representation

- `commandID`: Unique identifier for each command (Primary Key).
- `userID`: Foreign Key linking to the User who provided the command.
- `commandText`: The text representation of the voice command.

Relationships

- A **User** can create multiple **Floorplans**, indicating a one-to-many relationship.
- A **User** can also provide multiple **VoiceCommands**, indicating another one-to-many relationship.
- Each **Floorplan** can relate to multiple **VoiceCommands**, allowing tracking of commands associated with each generated floorplan.